影像處理實習

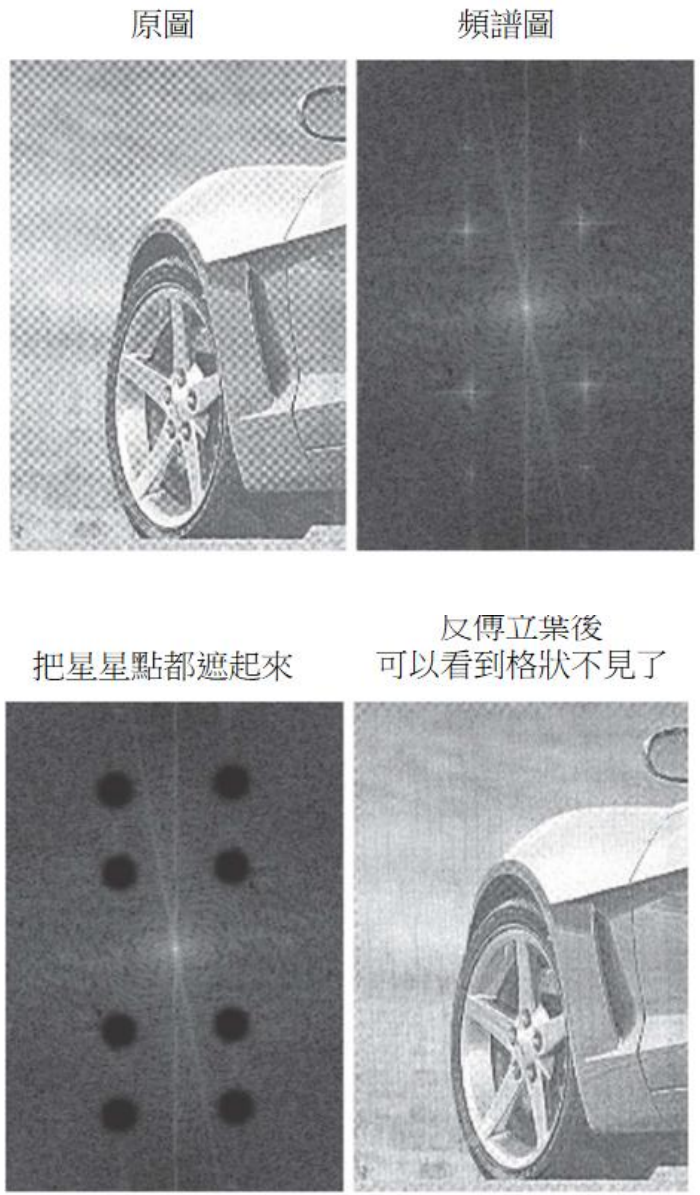
Week 9. 頻率域影像

助教 陳立堯

Part 0. 回顧

為何需要將影像轉成頻率域處理?

- 可以鎖定影像中的特定成分，如格狀點、波紋等
- 分析影像特性 (低頻 -> 模糊、平滑 | 高頻 -> 細節、雜訊)
- 壓縮影像



Part 0. 操作過程

圖像 → DFT → 頻率濾波 → IDFT → 濾波後圖像

傅立葉轉換

Part 1. DFT

$$F(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) \cdot \boxed{e^{-j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}}$$

M：影像寬　N：影像高

轉換後結果 (複數)
|F(u,v)|：振幅，表頻率強度
U代表水平頻率
V代表垂直頻率

原始影像的像素值
空間座標 (x,y)

旋轉因子：

為正弦波與餘弦波的組合，
涵蓋從(0, 0) ~ (M-1, N-1)
之間的可能性

Part 1. DFT

```
def dft2(image):
    N, M = image.shape
    dft = np.zeros((N, M), dtype=complex)
    for u in range(N):
        for v in range(M):
            sum_value = 0.0
            for x in range(N):
                for y in range(M):
                    angle = -2j * np.pi * ((u * x / N) + (v * y / M))
                    sum_value += image[x, y] * np.exp(angle)
            dft[u, v] = sum_value
    return dft
```

$$F(u,v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) \cdot e^{-j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

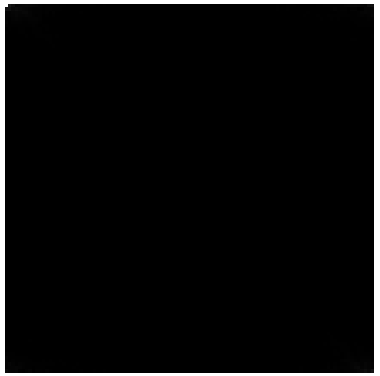
Part 1. 查看轉換後的頻譜圖

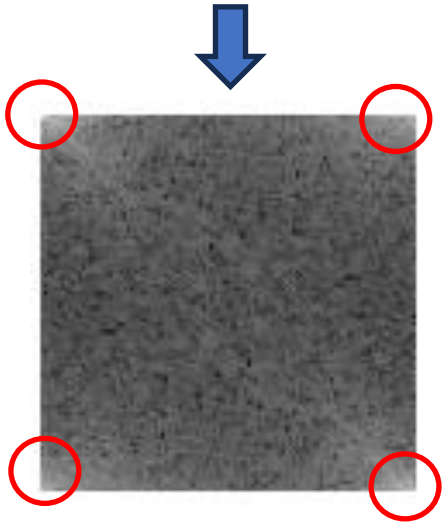
```
# 原始頻譜
plt.subplot(3, 3, 2)
plt.imshow(np.log(np.abs(F_uv) + 1), cmap='gray')
plt.title("Original Frequency Spectrum")
plt.axis('off')
```

取振幅 → Log轉換

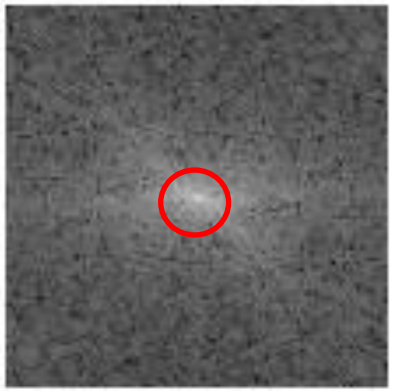
Log轉換：
拉伸低亮度
(100 -> 2)

壓縮高亮度
(10000000 -> 7)





```
# 中心反轉頻譜
plt.subplot(3, 3, 3)
plt.imshow(np.log(np.abs(centered_F_uv) + 1), cmap='gray')
plt.title("Centered Frequency Spectrum")
plt.axis('off')
```



頻率濾波

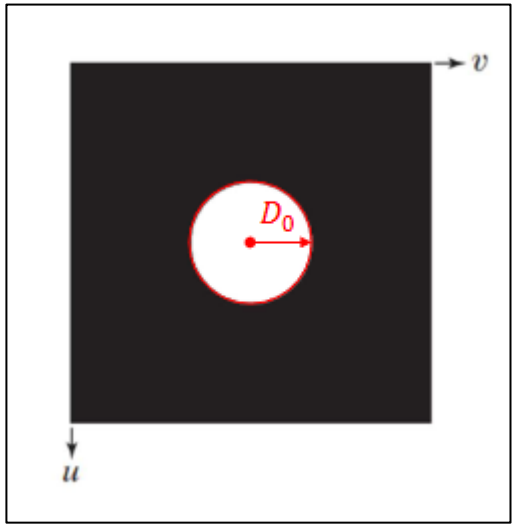
Part 2. 理想低通濾波器

$$H(u,v) = \begin{cases} 1 & ,若 D(u,v) \le D_0 \\ 0 & ,若 D(u,v) > D_0 \end{cases}$$

$$D(u,v) = \sqrt{\left(u - \frac{P}{2}\right)^2 + \left(v - \frac{Q}{2}\right)^2}$$

P=寬 , Q=高

```python
def ideal_low_pass_filter(shape, D0): #設定半徑為D0的圓圈
    N, M = shape
    H = np.zeros((N, M))
    center_x, center_y = N // 2, M // 2
    for u in range(N):
        for v in range(M):
            D_uv = np.sqrt((u - center_x) ** 2 + (v - center_y) ** 2)
            if D_uv <= D0:
                H[u, v] = 1
    return H
```



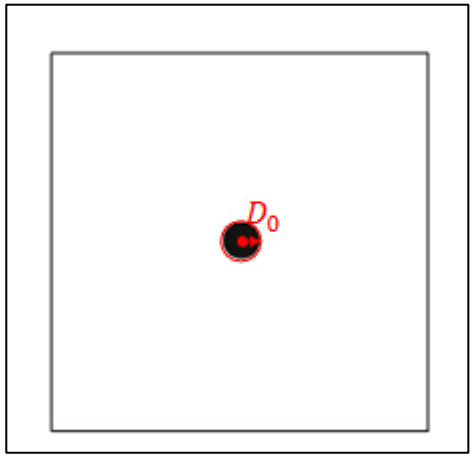
Part 2. 理想高通濾波器

```python
def ideal_high_pass_filter(shape, D0):
    N, M = shape
    H = np.ones((N, M))  # 預設全為1
    center_x, center_y = N // 2, M // 2
    for u in range(N):
        for v in range(M):
            D_uv = np.sqrt((u - center_x) ** 2 + (v - center_y) ** 2)
            if D_uv <= D0:
                H[u, v] = 0  # 圓內設為0
    return H
```

$$H(u,v) = \begin{cases} 0 & ,若 D(u,v) \le D_0 \\ 1 & ,若 D(u,v) > D_0 \end{cases}$$

$$D(u,v) = \sqrt{\left(u - \frac{P}{2}\right)^2 + \left(v - \frac{Q}{2}\right)^2}$$

P=寬，Q=高



反傅立葉

Part 3. IDFT

```
#f(x,y) = (1/NM) Σ Σ F(u,v) × e^(+2πj[(ux/N) + (vy/M)])
def idft2(dft):
    N, M = dft.shape
    idft = np.zeros((N, M), dtype=complex)
    for x in range(N):
        for y in range(M):
            sum_value = 0.0
            for u in range(N):
                for v in range(M):
                    angle = 2j * np.pi * ((u * x / N) + (v * y / M))
                    sum_value += dft[u, v] * np.exp(angle)
            idft[x, y] = sum_value / (N * M)
    return np.abs(idft)
```

補充. NP相關套件

```python
#使用 NumPy 的 2D FFT
def fft2_numpy(image):
    return np.fft.fft2(image)

#使用 NumPy 的 2D IFFT
def ifft2_numpy(spectrum):
    return np.fft.ifft2(spectrum).real

#將頻譜中心化(NumPy 內建)
def fftshift(spectrum):
    return np.fft.fftshift(spectrum)

#反中心化(濾波後需要)
def ifftshift(spectrum):
    return np.fft.ifftshift(spectrum)
```

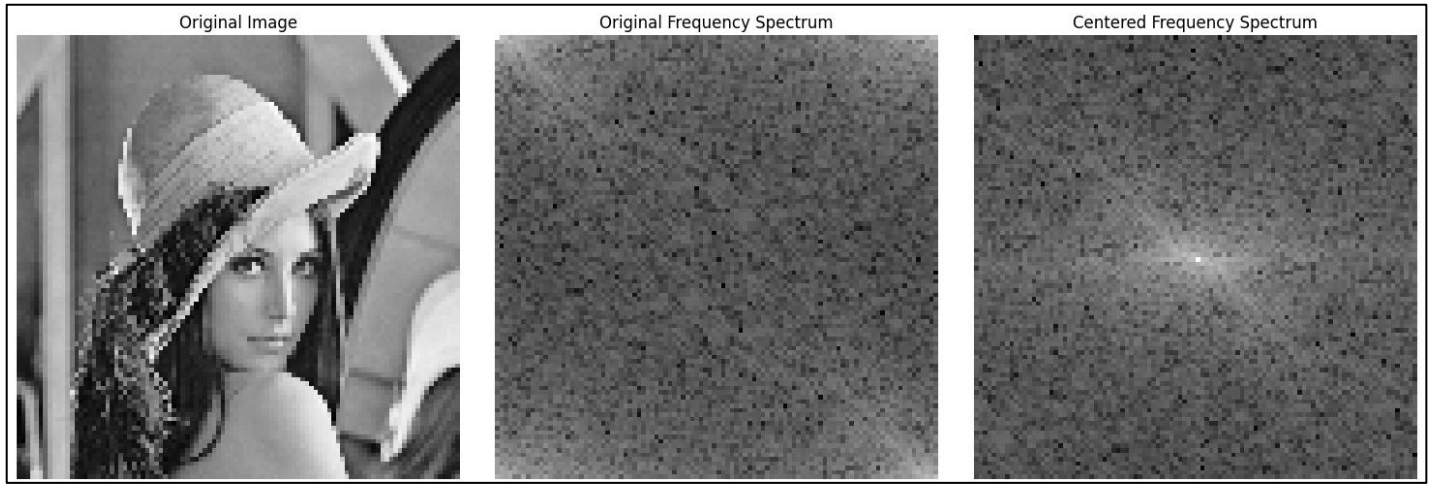
1D -

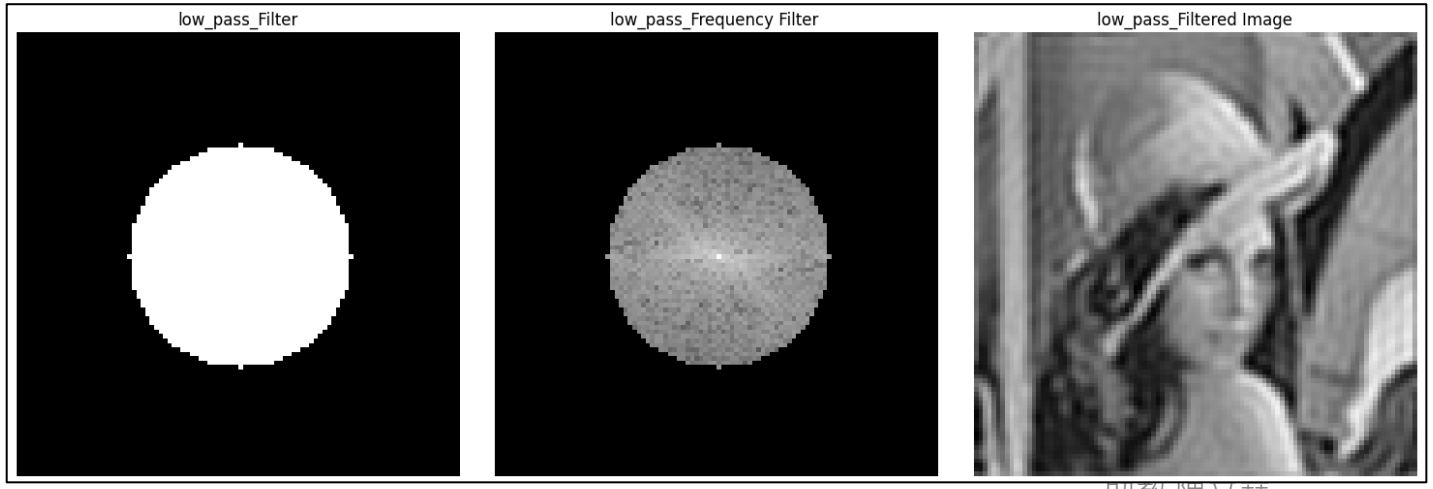
$$o(N^2) \rightarrow oN(logn)$$

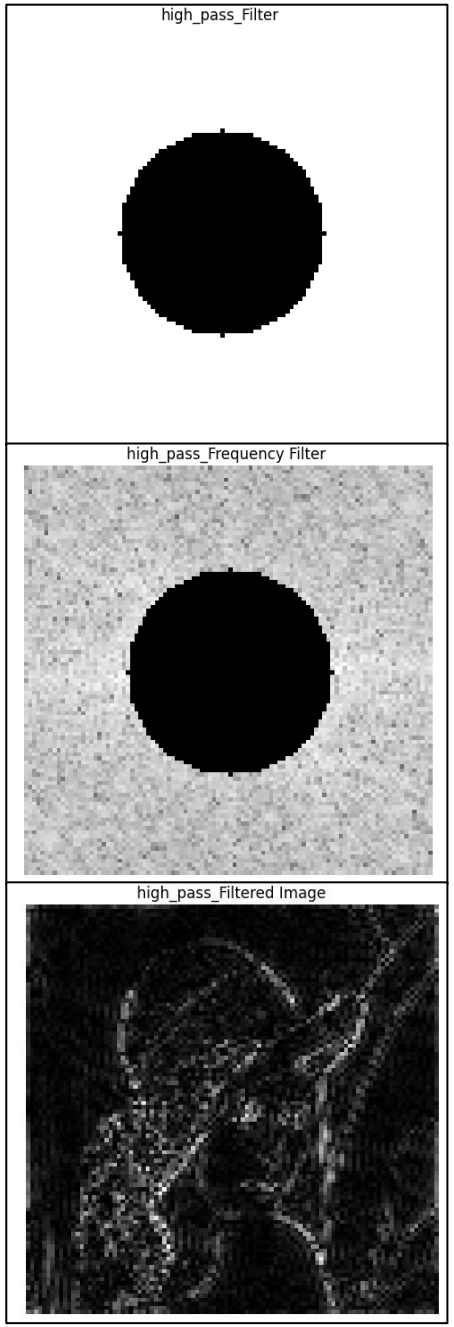
2D -

$$o(N^4) \rightarrow oN^2(logn)$$

補充. 結果說明


Original Image
Original Frequency Spectrum
Centered Frequency Spectrum


low_pass_Filter
low_pass_Frequency Filter
low_pass_Filtered Image


high_pass_Filter
high_pass_Frequency Filter
high_pass_Filtered Image

本日無作業

- 期中考週